

## Article

# MobileNets Can Be Lossily Compressed: Neural Network Compression for Embedded Accelerators

Se-Min Lim  and Sang-Woo Jun \* Computer Science Department, Donald Bren School of Information and Computer Sciences,  
University of California, Irvine, CA 92697, USA; seminl1@ics.uci.edu

\* Correspondence: swjun@ics.uci.edu

**Abstract:** Although neural network quantization is an imperative technology for the computation and memory efficiency of embedded neural network accelerators, simple post-training quantization incurs unacceptable levels of accuracy degradation on some important models targeting embedded systems, such as MobileNets. While explicit quantization-aware training or re-training after quantization can often reclaim lost accuracy, this is not always possible or convenient. We present an alternative approach to compressing such difficult neural networks, using a novel variant of the ZFP lossy floating-point compression algorithm to compress both model weights and inter-layer activations and demonstrate that it can be efficiently implemented on an embedded FPGA platform. Our ZFP variant, which we call ZFPe, is designed for efficient implementation on embedded accelerators, such as FPGAs, requiring a fraction of chip resources per bandwidth compared to state-of-the-art lossy compression accelerators. ZFPe-compressing the MobileNet V2 model with an 8-bit budget per weight and activation results in significantly higher accuracy compared to 8-bit integer post-training quantization and shows no loss of accuracy, compared to an uncompressed model when given a 12-bit budget per floating-point value. To demonstrate the benefits of our approach, we implement an embedded neural network accelerator on a realistic embedded acceleration platform equipped with the low-power Lattice ECP5-85F FPGA and a 32 MB SDRAM chip. Each ZFPe module consumes less than 6% of LUTs while compressing or decompressing one value per cycle, requiring a fraction of the resources compared to state-of-the-art compression accelerators while completely removing the memory bottleneck of our accelerator.

**Keywords:** embedded FPGA accelerators; compression; neural networks



**Citation:** Lim, S.-M.; Jun, S.-W. MobileNets Can Be Lossily Compressed: Neural Network Compression for Embedded Accelerators. *Electronics* **2022**, *11*, 858. <https://doi.org/10.3390/electronics11060858>

Academic Editor: Maciej Ławryńczuk

Received: 1 February 2022  
Accepted: 1 March 2022  
Published: 9 March 2022

**Publisher's Note:** MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.



**Copyright:** © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

## 1. Introduction

A great amount of research effort has focused on bringing the benefits of deep neural networks to resource-constrained embedded devices in order to support increasingly popular applications, such as the internet of things (IoT) or cyber-physical systems (CPS). Efforts include architectural solutions, including the use of power-efficient, high-performance accelerators, such as mobile GPUs [1] or FPGAs [2,3], as well as algorithmic solutions, such as designing compact neural networks with fewer resource requirements [4–6].

An imperative technology for embedded neural networks is deep compression [7], where a complex neural network can be compressed by *pruning* less significant edges, as well as *quantizing* weights and activations from floating-point values to smaller fixed-point values. Quantizing is an especially important technique. Smaller fixed-point values can not only reduce the memory size and bandwidth requirements, but also replace costly floating-point arithmetic with simpler fixed-point ones. Many important neural network models demonstrated maintaining high accuracy with aggressive quantization to 8-bit integers and beyond [8], and many hardware and software environments are optimized for such quantized models [9–12].

However, not all models can be quantized effectively, and aggressive quantization sometimes results in an unacceptably sharp accuracy decline. A prominent example is